

Nomes: Ana Júlia, João Lavina e Nicolý K. L. Araujo

Relatório Trabalho Final
Classificação de Manobra de Bicicleta 360

Processamento de imagens
Blumenau 2026

1. Introdução

Este projeto implementa um sistema de detecção e classificação de uma manobra de bicicleta em vídeos, combinando duas tecnologias de ponta em visão computacional: *YOLOv8n* [1] para detecção de objetos em tempo real e *TensorFlow/Keras* [2] para classificação da manobra específica. O sistema é capaz de processar vídeos contendo bicicletas, identificar quando uma bicicleta está presente no quadro e classificar se a bicicleta está realizando uma manobra de 360 graus ou está em posição normal.

2. Base de dados

A base de dados foi estruturada em sequências temporais de frames para viabilizar a classificação de manobras. Como o giro de 360° é um movimento contínuo e rotacionado, sua identificação exige a análise de múltiplos frames consecutivos, um único quadro isolado não contém informação suficiente para distinguir uma manobra de uma pedalada normal. Para isso, cada vídeo foi processado pelo detector *YOLO*, que localiza e recorta a região da bicicleta em cada frame. Os recortes resultantes foram agrupados em janelas deslizantes de 16 frames subamostrados ao longo de 48 frames de origem, formando as sequências de entrada para o modelo *CNN-LSTM*, que combina características de aparência e movimento para classificar a manobra.

3. Processo de Coleta e Preparação

Os dados foram coletados por meio da gravação de vídeos de bicicletas em dois cenários distintos: a execução da manobra de 360° e a pedalada normal. Ao todo, o dataset conta com 30 vídeos de manobras 360 e 30 vídeos de pedalada normal, capturados em diferentes ângulos e condições de iluminação. Os vídeos foram armazenados em formato *.mp4* no diretório *videos*.

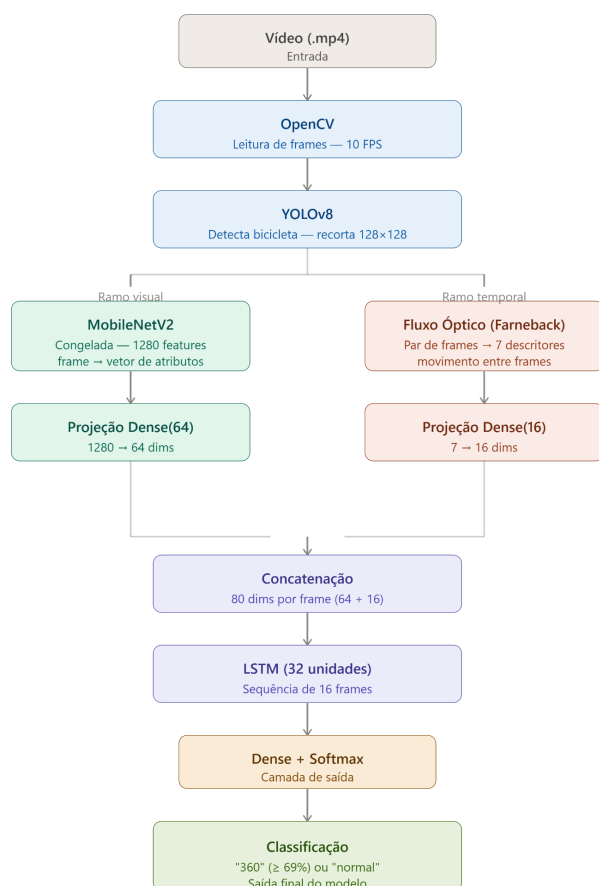
Leitura dos frames: Utilizando a biblioteca *OpenCV*, cada vídeo é lido quadro a quadro. Para reduzir redundância e adequar o volume de dados ao tamanho reduzido do dataset, é aplicada uma subamostragem temporal, limitando a taxa de extração a 10 FPS.

Detecção e recorte da bicicleta: Em cada frame selecionado, o modelo *YOLOv8n*, biblioteca *Ultralytics*, pré-treinado no dataset *COCO*, detecta a bicicleta na cena. A região da *bounding box* é recortada com uma margem de 15% e redimensionada para 128×128 pixels. Quando a bicicleta não é detectada, o sistema utiliza como *fallback* a detecção da classe "*person*".

Organização em sequências: Os recortes são salvos como sequências ordenadas de imagens em subpastas do diretório *sequences_dataset*, separadas por classe (360 e normal). A classe de cada sequência é inferida automaticamente a partir do nome do arquivo de vídeo. Cada amostra de

entrada do modelo consiste em uma janela de 16 frames subamostrados ao longo de 48 frames de origem (1 a cada 3), garantindo que cada amostra cubra a duração completa de uma manobra.

4. Arquitetura da Rede



O modelo de classificação recebe duas fontes de informação para decidir se a bicicleta está fazendo uma manobra de 360 ou apenas andando normalmente: a imagem de cada frame e o movimento entre frames consecutivos.

Uma janela deslizante com passo de 12 frames é utilizada para gerar múltiplas amostras a partir de cada sequência, funcionando como uma forma de *data augmentation* temporal.

Aparência: Cada frame recortado da bicicleta é passado por uma rede chamada *MobileNetV2* [3], que já vem treinada para reconhecer objetos em imagens. Ela transforma cada imagem em um resumo numérico de 1.280 valores que descrevem o que aparece no frame, posição do ciclista, ângulo da bicicleta, fundo, etc.

Movimento: Entre cada par de frames consecutivos, o sistema calcula o chamado fluxo óptico [4], basicamente, para onde cada pixel se moveu de um quadro para o outro. Esse campo de movimento é resumido em 7 números que indicam a intensidade do movimento, se há rotação (giro, como no 360) ou apenas deslocamento em linha reta (como na pedalada normal).

Fusão e decisão ao longo do tempo: Os dois resumos (imagem + movimento) são combinados e alimentados a uma rede do tipo *LSTM* [5], que analisa a sequência completa de 16 frames. A *LSTM* consegue "lembrar" o que aconteceu nos frames anteriores e perceber o padrão de rotação ao longo do tempo. No final, o modelo mostra a probabilidade de ser uma manobra 360 ou pedalada normal.

5. Testes do modelo

A validação do modelo foi feita de forma rigorosa utilizando validação cruzada agrupada por vídeo. Isso significa que sequências extraídas de um mesmo vídeo nunca aparecem simultaneamente no conjunto de treino e no de teste, garantindo que o modelo seja avaliado pela sua capacidade de generalizar para vídeos que ele nunca viu, e não apenas por memorizar trechos de vídeos já conhecidos.

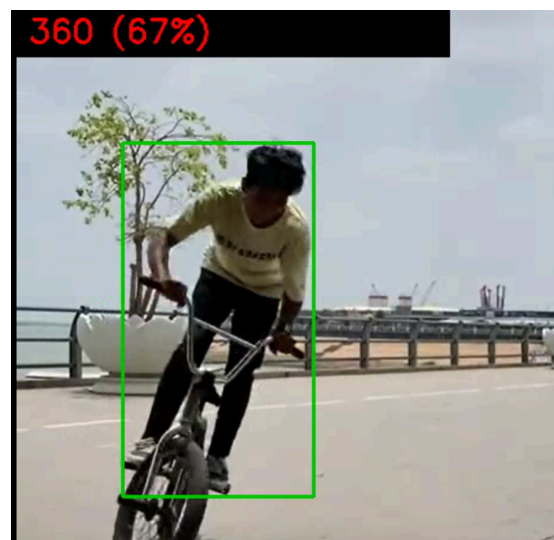
- O vídeo é lido frame a frame pelo *OpenCV*.
- O *YOLO* detecta e recorta a bicicleta em cada frame.
- Quando o buffer acumula frames suficientes para cobrir uma manobra inteira (48 frames de origem, subamostrados em 16), o modelo *CNN-LSTM* faz a predição. A classificação só é exibida como "360" quando a confiança do modelo atinge pelo menos 69% — caso contrário, é tratada como "normal". Isso reduz falsos positivos.
- O resultado é desenhado sobre o vídeo em tempo real (*bounding box* + rótulo com porcentagem de confiança).

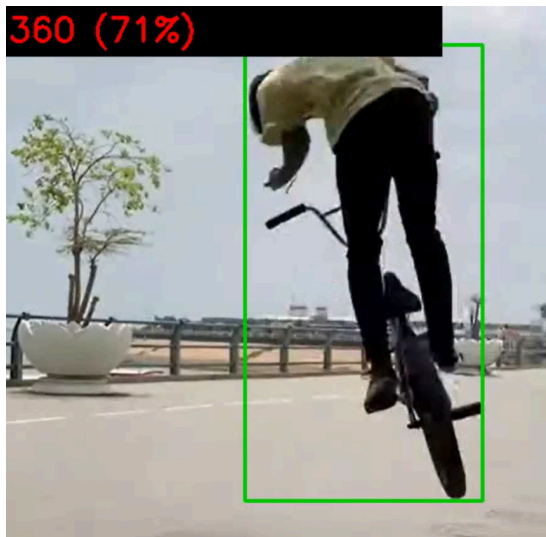
O modelo foi testado em vídeos de manobras 360 gravados de diferentes ângulos (de frente, de lado, etc.). O modelo reconhece corretamente a manobra de 360 com confiança entre 52% e 99%. Vídeos de pedalada normal são classificados como "normal" na maioria dos frames. O principal desafio

encontrado foi a generalização da classe "normal", uma vez que o modelo confundia o 360 com outras manobras.



Devido também a má separação dos frames e o treinamento do modelo em cima de um dataset 'envenenado', foi obtido a detecção precoce da manobra, pois os frames de antes da manobra também estavam contidos no dataset que treinava a detecção de manobra.





6. Código Fonte

OpenCV

Responsável por ler os frames do vídeo, redimensiona os recortes, converte cores, calcula o fluxo óptico (deslocamento dos pixels entre frames) para detectar rotação, e grava o vídeo final com as anotações desenhadas na tela.

Ultralytics (YOLOv8)

Faz a detecção da bicicleta em cada frame. O modelo já vem treinado para reconhecer objetos comuns (COCO), então basta passar o frame e ele retorna a posição da bicicleta na imagem. Com essa posição, recortamos apenas a região da bicicleta para alimentar o classificador.

TensorFlow / Keras

Constrói e treina a rede neural que classifica a manobra. Usa a *MobileNetV2* para extrair características visuais de cada frame, e uma *LSTM* para analisar a sequência de

16 frames ao longo do tempo e perceber o padrão de rotação do 360. No final, dá a probabilidade de ser uma manobra ou pedalada normal.

NumPy

Faz todos os cálculos matemáticos com matrizes, como transformar imagens em números, calcular médias e desvios do fluxo óptico e organizar os dados no formato que o modelo espera.

Scikit-learn

Usada para avaliar o modelo de forma justa e para calcular pesos que compensam o fato de ter mais vídeos de 360 do que de normal no dataset.

Matplotlib

Gera o gráfico de treinamento que mostra a evolução do erro e da acurácia ao longo das épocas, permitindo visualizar se o modelo está aprendendo corretamente.

7. Discussão e resultados obtidos

Durante o desenvolvimento do sistema de detecção da manobra 360, um dos primeiros obstáculos enfrentados foi a perda significativa de frames durante a etapa de reconhecimento da bicicleta. Essa perda ocorre porque, em determinados momentos do vídeo, o modelo de detecção utilizado não consegue identificar o objeto com confiança

suficiente. Para contornar esse problema, foi pensado uma estratégia de interpolação das posições da bicicleta entre os frames em que a detecção foi bem-sucedida. Dessa forma, mesmo quando o detector falha em um ou mais frames consecutivos, o sistema consegue estimar a posição provável do objeto com base nos pontos conhecidos antes e depois da falha, mas não foi necessário.

O segundo ponto relevante diz respeito à estratégia de classificação. Como o objetivo do projeto é identificar especificamente a manobra 360, seria insuficiente treinar o modelo apenas com duas classes simplistas, "andando normal" e "fazendo 360", porque outras manobras distintas do 360, como saltos, empinadas ou giros parciais, poderiam ser erroneamente classificadas como 360 caso o modelo não tivesse sido exposto a essa diversidade. Portanto, foi tomada a decisão de enriquecer a classe "normal" com exemplos de outras manobras e variações de pilotagem, de modo que o classificador aprenda que "tudo que não é 360 é normal", mesmo que visualmente apresenta movimentos complexos. Isso força o modelo a ser mais restritivo na sua decisão de classificar algo como 360, exigindo que ele identifique as características realmente específicas dessa manobra.

Tipo de imagem utilizado no dataset normal:



Recortes iniciais que precisaram ser removidos do dataset, pois como estava muito recortado e devido a isso não estava sendo possível comparar a posição da bicicleta ao chão, e isso compromete o treinamento do modelo:



8. Conclusão

O trabalho apresentou o desenvolvimento de um sistema de visão computacional capaz de classificar manobras de bicicleta do tipo 360° a partir de vídeos, utilizando uma pipeline composta por detecção de objetos, extração de características visuais, análise de movimento por

fluxo óptico e classificação temporal por rede *LSTM*.

O sistema demonstrou capacidade de identificar corretamente a manobra de 360° na maioria dos cenários testados, com confiança variando entre 52% e 99%. A abordagem de duas entradas, combinando aparência e movimento, mostrou-se adequada para o problema, uma vez que a manobra depende essencialmente da dinâmica temporal e não apenas de um quadro isolado.

Como trabalho futuro, recomenda-se a ampliação do dataset com mais vídeos da classe normal, a inclusão de outros tipos de manobras e a exploração de modelos de detecção mais recentes, visando melhorar a acurácia e a aplicabilidade do sistema em cenários reais.

9. Referências Bibliográficas

- [1] JOCHER, G. et al. YOLOv8 by Ultralytics. 2023. Disponível em: <https://github.com/ultralytics/ultralytics>. Acesso em: 26 jun. 2026.
- [2] ABADI, M. et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. 2016. Disponível em: <https://www.tensorflow.org/>. Acesso em: 26 jun. 2026.
- [3] SANDLER, M. et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.
- [4] LUCAS, B. D.; KANADE, T. An Iterative Image Registration Technique with an Application to Stereo Vision. In: Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI), 1981.
- [5] DONAHUE, J. et al. Long-term Recurrent Convolutional Networks for Visual Recognition and Description. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017.